Vector Clocks and Hybrid Logical Clocks

detecting concurrent updates

Phase 4 videos on YouTube

In one line: a vector clock keeps one counter per node, so comparing two versions tells you whether one came after the other or whether they were concurrent and conflict; hybrid logical clocks combine physical time with a counter to get timestamps close to real time that still respect causality.

1 The disappearing cart item

Two users add items to the same shopping cart, on two different replicas, at the same moment. Neither update saw the other. With last-write-wins, one item silently disappears. To merge correctly, the system must first **know** that the two versions were concurrent. Lamport clocks (lesson 6) can't tell that; vector clocks can.

2 How a vector clock works

Each version of a value carries a **vector**: one counter per node, like `[X:2, Y:1]`. When node X updates the value, it increments its own entry. When versions meet, compare them entry by entry:



Figure 1. Comparing two vector clocks gives one of three answers. The third one is the conflict a Lamport clock can't see.

When a node receives a version, it takes the element-wise **maximum** with what it has. So **a → b if and only if V(a) < V(b)**: vector clocks capture causality exactly.

WORKED EXAMPLE: DYNAMO'S SHOPPING CART

Replica X stores a cart with a book at version `[X:1]`. During a partition, one session adds headphones on X, giving `[X:2]`. Another session reads the old cart and adds a charger on replica Y, giving `[X:1, Y:1]`. When the partition heals, neither vector is greater: X has more for X, Y has more for Y. So the store keeps both as **siblings** and returns both on the next read. The cart logic merges them by union, and writes `[book, headphones, charger]` at `[X:2, Y:1]`, which now dominates both. (A union can bring back a deleted item; Dynamo accepted that trade-off.)

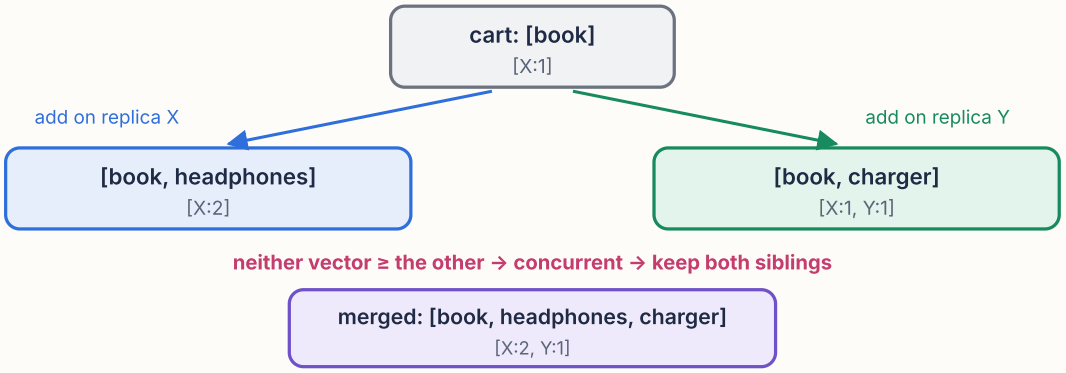


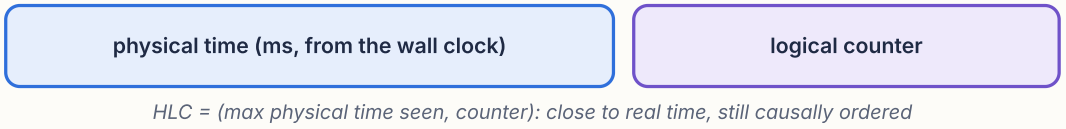
Figure 2. Concurrent versions are detected and kept, then merged by the application.

3 Costs and variants

The vector grows with the number of nodes (or clients) that write. Real systems use **version vectors** per replica rather than per client, prune old entries, or use **dotted version vectors** (Riak) to avoid false conflicts. Many stores give up and use last-write-wins for simplicity; that's fine only when losing a concurrent write is acceptable.

4 Hybrid logical clocks

Sometimes you want timestamps that are **close to real time** (for 'as of' reads, TTLs and debugging) and still respect causality, without TrueTime hardware. A **hybrid logical clock** (HLC) is a pair: the largest physical time seen so far, plus a small logical counter that breaks ties and covers clock lag. On send or receive it takes the max of local physical time and the received HLC, bumping the counter when physical time hasn't moved. CockroachDB and YugabyteDB use HLCs together with a maximum allowed clock offset.



HLC = (max physical time seen, counter): close to real time, still causally ordered

Figure 3. An HLC fits in 64 bits and can be compared like a timestamp.

COMMON MISTAKES

- Using last-write-wins where concurrent writes must both survive.
- Per-client vector entries that grow without bound.
- Detecting conflicts but having no merge rule for the application.
- Treating an HLC as exact physical time.

5 Key terms

Vector clock: one counter per node; detects concurrency

Siblings: concurrent versions kept until merged

Hybrid logical clock: physical time + logical counter

Version vector: vector clock attached to data versions per replica

Dominates: every entry ≥ and at least one >

6 Interview questions

Q1. What can vector clocks tell you that Lamport clocks can't? BASIC

Whether two events were concurrent. Comparing vectors shows if one happened before the other or neither did.

Q2. How do you compare two vector clocks? BASIC

If every entry of A is ≤ B's (and they differ), A happened before B. If it's the other way round, B happened before A. Otherwise they're concurrent.

Q3. Two replicas accepted writes to the same key during a partition. How do you avoid losing one? SCENARIO

Track versions with vector clocks, detect that they're concurrent, keep both as siblings, and merge them with an application rule or a CRDT.

Q4. Your vector clocks are growing huge. Why, and what do you do? SCENARIO

Entries are per client or per short-lived node. Use per-replica version vectors, prune old entries, or switch to dotted version vectors.

Q5. Why do CockroachDB and YugabyteDB use HLCs instead of vector clocks? TRICKY

They need one compact, comparable timestamp per transaction that's close to real time for MVCC and 'as of' reads; HLCs give that while respecting causality, given a bound on clock offset.

Q6. Does a vector clock merge the data for you? TRICKY

No. It only tells you there's a conflict. The application or a CRDT has to decide the merged value.

Next: SD-H-008 · Leader Election backendroadmap.com